Building AI Threat-Modeling and Red-Teaming Capability with MITRE ATLAS

MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) is a public knowledge base of adversary tactics and techniques against AI/ML systems, modeled after ATT&CK. Organizing your adversarial test suite by ATLAS **tactic** (the kill-chain stage) makes coverage gaps visible — you can see which stages you test and which you don't.

From threat model to test suite

1. **Diagram the AI/ML dependency chain:** data sources → training/fine-tune → model artifact → serving API → RAG/index → agent/tools → downstream actions.
2. **Map techniques to each link.** For every link, list applicable ATLAS techniques (poisoning at data, extraction at the API, injection at serving, etc.).
3. **Author a runnable case per technique** with a detector that decides success.
4. **Run in CI and track ASR** so regressions fail the build.

ATLAS techniques covered in this repo

`airte.atlas.ttp_suite` maps techniques to runnable `airte.redteam` cases and to the OWASP LLM Top 10. Coverage spans eight tactics from Reconnaissance through Impact:

ATLAS ID	Technique	Tactic	OWASP	Cases
AML.T0051	LLM Prompt Injection	Initial Access/Exec	LLM01	yes
■ .000	Prompt Injection: Direct	Initial Access/Exec	LLM01	yes
■ .001	Prompt Injection: Indirect	Initial Access/Exec	LLM01	yes
AML.T0054	LLM Jailbreak	Defense Evasion	LLM01	yes
AML.T0020	Poison Training/Index Data	Resource Dev	LLM04	yes
AML.T0056	Extract LLM System Prompt	Exfiltration	LLM07	yes
AML.T0024	Exfiltration via Inference API	Exfiltration	LLM02	yes

AML.T0048	Model Inversion Harm	Impact	LLM02	yes
AML.T0029	Denial of ML Service	Impact	LLM10	yes
AML.T0018	Manipulate AI Model (supply)	Persistence	LLM03	yes
AML.T0040	Discover AI Model / Tooling	Reconnaissance	LLM07	yes
AML.T0043	Craft Adversarial Data / Stage	ML Attack Staging	LLM01	yes

> Verify exact ATLAS IDs/names against the current matrix at atlas.mitre.org when reporting — the framework evolves, and sub-technique IDs (e.g. AML.T0051.000 direct / .001 indirect) are added over time.

```
from airte.atlas import build_atlas_suite, map_owasp_to_atlas
suite = build_atlas_suite()          # {atlas_id: [AttackCase, ...]}
map_owasp_to_atlas("LLM01")          # techniques tied to prompt injection
```

```
python -m airte.atlas.ttp_suite      # prints coverage report
```

Building the adversarial test suite

- **Start from real ingress.** Your highest-value cases mirror how attackers reach your system: indirect injection via the documents/tools your agent actually reads.
- **Detectors over eyeballing.** Each case ships a programmatic success detector (canary leak, poison marker absorbed, refusal bypassed) so runs are objective and CI-gatable.
- **Cover the whole chain.** Don't only test serving-time injection; add poisoning (data/index) and extraction (API) cases so you cover Resource Development and Exfiltration tactics, not just Initial Access.
- **Sub-technique granularity.** Prompt injection is split into Direct (`AML.T0051.000`) and Indirect (`AML.T0051.001`) via `prompt_injection.direct_cases()` / `indirect_cases()`, so you can report coverage at sub-technique resolution.
- **All techniques ship runnable cases.** Prompt-level suites live in `airte.redteam` (`unbounded_consumption`, `supply_chain`, `model_inversion` were added to complete coverage); artifact-level supply-chain auditing lives in `airte.supply_chain`. Extend each with environment-specific cases.

Operationalizing

- Wire `airte.redteam.harness` into CI with an ASR threshold gate.
- Track ASR per ATLAS tactic over time; a rising ASR after a prompt/model/tool change is a regression signal.
- Feed findings back into the threat model and the secure-by-default checklist, then add a regression case for each fix.